Development and evaluation of a modular speaker diarization pipeline

Reinhold Brant

DHBW Stuttgart
Stuttgart, Germany

***Abstract*—later**

***Index Terms*—Scientific writing, Typesetting, Document creation, Syntax**

I. Introduction

Speaker diarization describes the task of deciding who is speaking at a given moment of time. To achieve that, there are multiple steps involved to process an audio input and calculate data on the amount of speakers and who is speaking when. The aim of this work ist to implement a modular diarization pipeline. Furthermore, we want to evaluate different hyperparameter settings and models to achieve a DER (diarization error rate) of below 0.2, which will be seen as achievable for building a modular pipeline from pretrained models as well as tuning hyperparameters. State-of-the-art pipelines achieve DER rates under 0.1.

A. Setup

As a first step, the project is set up with uv, which is a modern Python package manager to handle package installation and Python versioning. To test out different pipelines efficiently, we will implement Python base classes for modules, which will hold different functionalities of the pipeline and also implement pipeline classes, that will be used simplify execution of different pipelines sequentially. This will lead to a more efficient data collection process.

B. Dataset

To evaluate the implemented pipelines a sufficient dataset is needed. In this work, the AMI Corpus dataset will be used. This dataset is "a multi-modal data set consisting of 100 hours of meeting recordings" [1]. It consists of different meetings with recordings for every speaker per meeting and an audio mix containing the mix of all speakers of the meeting, which will be used in this work. In addition to that, the dataset provides reference data as XML files, which will be used to calculate the DER against our pipeline hypotheses.

C. Metrics

To evaluate implemented pipelines, the DER (diarization error rate) will be used, as it is the standard evaluation metric of diarization systems [2]. The DER is calculated with the following formula:

$$\mathrm{DER} = \frac{\text{false alarm} + \text{missed detection} + \text{confusion}}{\text{total}} \quad (1)$$

False alarms are cases, where the system decides that any speaker is currently speaking when in fact there is silence in the audio. Missed detections are cases, where the system decides that there is silence when in fact there is speech. Confusion is the wrong classification of a speaker. The meaning of the DER calculation segments is visualized in Fig. 1.

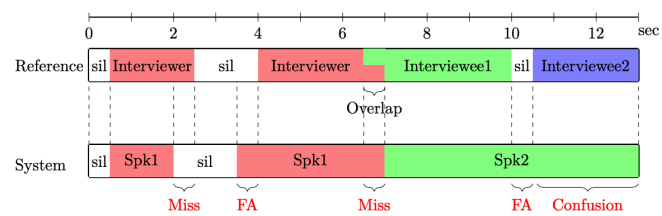

Fig. 1. DER calculation components [3]

II. Methods

A. Pipeline development

In order to achieve the aim of speaker diarization with a DER of below, we will choose an experimental approach. First, we will setup a fully functional pipeline, which will be gradually improved by means of testing alternative models and approaches for different steps of the pipeline. After each change the pipeline will be evaluated by means of calculating the DER. This process will be repeated in iterations until a sufficient DER is achieved for a the pipeline. After that, key components of the pipeline will be changed to alternative models oder algorithms to compare to the main pipeline. In the end, the implemented pipeline will be compared to other state-of-the-art methods in terms of DER.

B. Pipeline result visualization

To better understand the clustering results and validate the quality of speaker embeddings, visualization techniques are employed. The pipeline uses dimensionality reduction methods such as **Principal Component Analysis (PCA)** to project high-dimensional embedding vectors (typically 512-dimensional) into a 2D or 3D space suitable for visualization. PCA is computationally efficient and preserves global structure. The reduced embeddings are then plotted with color-coding by cluster assignment, allowing for visual inspection of cluster separation and identification of potential outliers or misclassified embeddings. This visualization helps identify

parameter tuning issues early in the development process and provides qualitative validation of the clustering performance before evaluating quantitative metrics like DER. In addition to that, the visualization includes the feature, to click on data points and listen to their respective audio chunk from the dataset for acoustic validation.

III. Initial Pipeline imnplementation

A. Basic architecture

To first setup a working diarization pipeline we will focus on a classic modular pipeline approach. This modular approach will consist of voice activity detection (VAD), feature extraction, clustering and post processing (see Fig. 2). The pipeline will take an audio path to the AMI mixed audio sample as input, run the pipeline on the audio and output a label array with annotations on which speaker spoke when. This array can be converted to RTTM (Rich Transcription Time Marked) format.

The following diagram shows the modular appraoch with which the pipeline will be implemented.

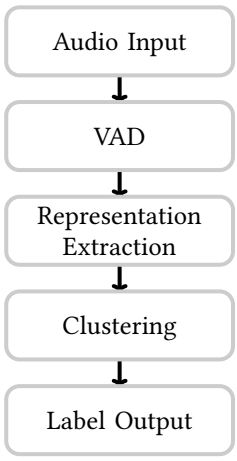

Fig. 2. Modular diarization pipeline

B. Voice Activity Detection

In order to detect who is speaking when, the system has to detect whether there is speech or silence. For that, a voice detection system can be used. There is a variety of VAD systems that are used in industry solutions. To determine a first method to setup a VAD module in the pipeline, we can compare different VAD solutions with help of a precision-recall diagram. The precision on the y-axis is determined by what percentage the model is correct when classifying an audio signal as speech. On the x-axis the recall is determined by how many of the actual positives where found by the model. Any point on a line in the diagram represents a threshold. By lowering the threshold the model will find nearly all positives but have a decrease in precision. By raising the threshold the model will find less false positives but miss some true positives (see Fig. 3). Among the possible solutions presented in the diagram we chose the Silero VAD module to setup a first pipeline prototype, as it has a good precision with higher recall values (see Fig. 3).

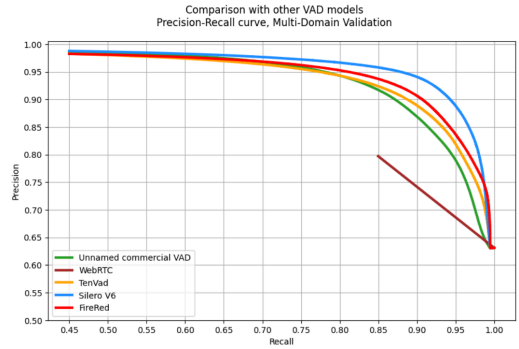

Fig. 3. VAD precision-recall diagram [4]

The Silero model can be installed by using uv add silero-vad (analog to pip usage) and loaded into the system with load_silero_vad(). To get the speech timestamp segments the function get_speech_timestamps() will be used. This method has a number of hyperparameters that are crucial to tune for good perfomance (see later). The output of the VAD model is an array of dictionaries with the following format [4]:

[{"start": time_start * SAMPLING_RATE, "end": time_start * SAMPLING_RATE}].

C. Convert segment to audio

Now that the VAD system predicted speech timestamps for a given audio, the system needs to cut the audio chunks from the original audio with respect to the calculated timestamps. This is done by converting the audio to a Numpy array and slicing it by the product of the **timestamp** and the **sampling rate**.

D. Feature extraction

In general, to categorize speakers the system needs to extract features from the speech chunks of the audio. These features will be used to build clusters. In order to build features from speech chunks, we will first use a pretrained Pyannote embedding model as it is quick to set up.

This model is an x-vector TDNN-based model to build so called embedding vectors from audio chunks. The TDNN achitecture of this model is based on deep neural networks. These are large mathematical models constructed of many “neurons” resembling the human brain [5]. In a classical setup extraction systems create a vector of a speakers voice based on the input audio (i-vectors) [6]. These i-vectors are built by using Gaussian-Mixture-Models (GMM) with matrix projection [6]. These vectors are a generalized representation of speech and can be imprecise. This can be improved by x-vectors, which are output of a DNN. The DNN is trained to discriminate between speakers directly. Ths way, the vector representations are more suitable for clustering [6]. A TDNN is a Time-Delay Neural Network, which also considers time based relationships between data to compute x-vectors. It looks at multiple points in time to build an embedding from a time window instead of a frame [6].

The Pyannote embedding extractor can be set up by loading a pretrained model from Huggingface [7]. It takes a Numpy array built from a wav-audio segment as input and outputs a 1×512 embedding vector. This is done for all audio segments built from the VAD timestamps. We get a $N \times 512$ x-vector array which can be used for clustering.

E. Clustering

Clustering algorithms group similar embedding vectors to assign them to individual speakers. Three common approaches for speaker diarization are:

DBSCAN groups embeddings based on local density. It identifies clusters by connecting points that are within distance ε of each other and requires at least min_samples points per cluster. Points that do not belong to any cluster are marked as noise. DBSCAN does not require specifying the number of clusters beforehand but can struggle with varying cluster densities [8].

Agglomerative Hierarchical Clustering iteratively merges the closest pairs of embeddings or clusters based on a linkage criterion (e.g., average, complete linkage). It produces a dendrogram that can be cut at different heights to obtain the desired number of clusters. This approach is stable and works well when the speaker count is known or estimated in advance [9].

Gaussian Mixture Models (GMM) model the embedding space as a mixture of Gaussian distributions, where each Gaussian represents a speaker. The model learns the mean, covariance, and mixing weights of each Gaussian through expectation-maximization (EM). GMM-based clustering provides probabilistic assignments and is particularly effective for speaker diarization, as it naturally captures the statistical properties of speaker embeddings [10]. These methods will be used to test which algorithm yields best results in the pipeline.

To cluster embedding vectors in the initial pipeline, DBSCAN will be used.

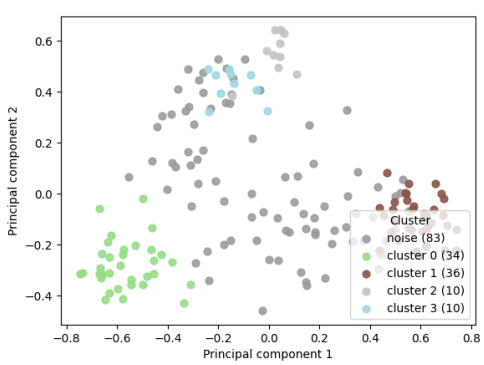

Fig. 4. VAD precision-recall diagram [4]

It can be observed that the majority of segments are classifed as noise. Furthermore upon further analysis, which is a qualitative analysis listening to the segments and their label, it becomes apparent, that this pipeline can be further optimized. The DER of this pipeline is dependant on the embedding and VAD variance but can be pinned down to DER ≈ 0.55, which is not sufficient.

IV. Pipeline improvement

In the initial imnplementation of the modular diarization pipeline it became apparent, that the quality of detections is not sufficient. In order to improve the pipeline, it is reasonable to start with the key data the rest of the modules depend on, which is the VAD segments.

A. Tuning VAD parameters

The initial paramters for the VAD system were as follows:

- **Threshold**: the higher the threshold the stricter the VAD system is with respect to what is detected as speech and what is detected as silence. This was set at **0.5**.
- **min_speech_duration**: Determines the minimum length of a speech segment. This was set at **400ms**.
- **max_speech_duration**: Determines the maximum length of the speech segment. This was set to **1.5s**.
- **min_silence_duartion**: Determines the minimum amount of silence between speech segments. Thsi was set to **500ms**.
- **speech_pad_ms**: This adds a padding before and after the segment to account for more context information. This was set to **20ms**.

After evaluating the initial pipeline it seemed reasonable to set **max_speech_duration** to **3 seconds**. This way, the system would not cut off speech chunks in the middle of an utterance. Otherwise, the embedding extraction would not produce consistent results. This improvement was made with the later idea to split detected segments with a sliding window approach to not yield too long segments. In addition to that, the min speech duration was reduced to **250ms**. This parameter change was tested with **DBSCAN** and **C-space** clustering and yielded following results:

TABLE I
DER comparison across different clustering methods and VAD tuning

Method	DER (initial)	DER (improved)
DBSCAN	0.55	0.49
C-Space (Agglomerative)	0.51	0.46

Although this did not result in a large improvement of the DER, we could reduce the DER, which was an improvement. All further improvements were made on the basis of C-Space clustering.

B. Merging segments

Merging adjacent VAD segments reduces fragmentation caused by brief pauses within a single speaker's turn. By combining consecutive segments of the same speaker into longer blocks, the embedding extraction process receives more context, leading to more stable and discriminative

speaker representations. We now ran a test to evaluate the DER with this change, which resulted in a DER decrease of around **3%**. Again, this change would not be sufficient to improve the pipeline significantly.

C. Sliding window

Instead of clustering entire VAD segments (which can be long and contain multiple speakers), the sliding window approach divides all segments into fixed-length overlapping windows (e.g., 1.5 seconds with 0.75-second step). This ensures that each embedding corresponds to a pure single-speaker utterance.

TABLE II
DER COMPARISON ACROSS DIFFERENT CLUSTERING METHODS WITH SLIDING WINDOW

Method	DER
C-Space (Agglomerative)	0.46
C-Space with Merging	0.42
C-Space with sliding window	0.43

In a further step, merging and sliding was combined to yield a DER ≈ 0.36, which was an improvement.

D. Second VAD parameter tuning

With sliding windows in place to ensure consistent, short segment length, the maximum speech duration of the VAD module was set to infinite in order to not have utterances cut off. This improved the DER significantly to a DER $\approx$ 0.22. Further reducing the step size of the sliding window to **0.5s** impoved the DER to **0.196**. In general, the pipeline was improved through a combination of architectural changes and tuning of parameters.

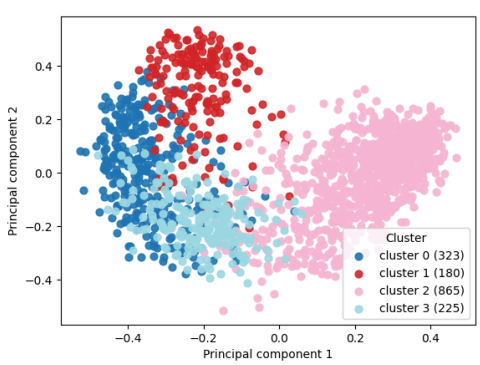

Fig. 5. DER 0.19 pipeline clusters with merging and sliding window

It can be oberseved that the clusters look more dense than with the initial pipeline. Moreover, sampling the data points acoustically works better.

V. Results

Although the initial modular pipeline did not perform according set standards, through architectural changes and tuning of hyperparameters, the DER of the modular pipeline could be decreased to DER ≈ 0.19, which is a satisfactory result for a modular pipeline without sophisticated overlap detection. Although VAD parameter tuning yielded small improvements, it was a combination of the sliding window approach with the tuning of hyperparameter values, that improved the pipeline the most (see Fig. 6).

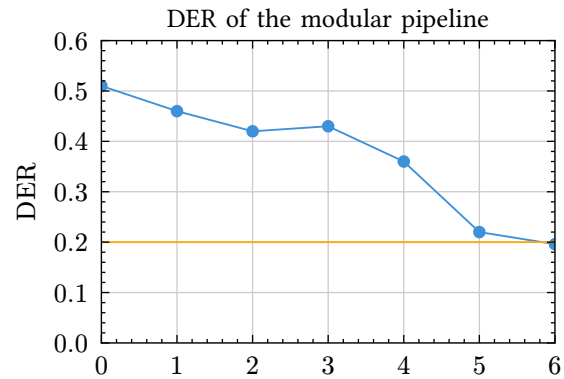

Fig. 6. DER improvement of the modular pipeline

References

[1] J. Carletta *et al.*, "The AMI Meeting Corpus." [Online]. Available: https://groups.inf.ed.ac.uk/ami/corpus

[2] H. Bredin, "pyannote.metrics: Diarization error rate (DER) reference documentation." [Online]. Available: https://pyannote.github.io/pyannote-metrics/reference.html#diarization

[3] F. Landini, "From Modular to End-to-End Speaker Diarization." [Online]. Available: https://arxiv.org/abs/2407.08752

[4] "Silero VAD GitHub repository." [Online]. Available: https://github.com/snakers4/silero-vad

[5] "Deep Neural Networks." [Online]. Available: https://www.iese.fraunhofer.de/de/trend/kuenstliche-intelligenz/deep-learning.html

[6] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, "X-Vectors: Robust DNN Embeddings for Speaker Recognition," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5329–5333. doi: 10.1109/ICASSP.2018.8461375.

[7] "Pyannote Embedding Huggingface." [Online]. Available: https://huggingface.co/pyannote/embedding

[8] "DBSCAN Scikit-learn." [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html

[9] "Agglomerative Clustering." [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.AgglomerativeClustering.html

[10] "Gaussian Mixture Models." [Online]. Available: https://scikit-learn.org/stable/modules/mixture.html